



EDUQA.PE

INTRODUCCIÓN A SQLITE CON PYTHON



SESIÓN 03

Consultas, agregación y JOIN

AUTOR

Alejandro Valentino Seminario Medina

AI Engineer



Filtrar con WHERE

La cláusula **WHERE** deja pasar solo las filas que cumplen una condición. Las condiciones se combinan con los operadores **AND** y **OR**.

```
cursor.execute(
    "SELECT nombre FROM productos WHERE precio > ? AND activo = ?",
    (100, 1),
)
print(cursor.fetchall())
```

Salida

```
[('Laptop',), ('Monitor',)]
```



Filtrar con WHERE

Documentación: [SELECT](#) · [Expresiones](#)



Comparar texto con LIKE

El operador **LIKE** compara una columna con un patrón. Tiene dos comodines, y cada uno se recorre por separado.

Nota técnica. LIKE no distingue mayúsculas de minúsculas, pero solo en los caracteres ASCII. Las letras acentuadas y la eñe sí se distinguen, salvo que SQLite se compile con la extensión ICU.

Documentación: [Expresiones](#)



El porcentaje

El signo de porcentaje (%) representa cualquier cantidad de caracteres, incluida ninguna.

```
cursor.execute("SELECT nombre FROM vendedores WHERE nombre LIKE ?", ("Ana%",))  
print(cursor.fetchall())
```

Salida

```
[('Ana Quispe',)]
```

Colocado a ambos lados, busca el fragmento en cualquier posición del texto.

```
cursor.execute("SELECT nombre FROM vendedores WHERE nombre LIKE ?", ("%pe%",))  
print(cursor.fetchall())
```



El porcentaje

Salida

```
[('Ana Quispe',)]
```

Documentación: [Expresiones](#)



El guion bajo

El guion bajo (_) representa exactamente un carácter, ni más ni menos.

Nota técnica. Para buscar un porcentaje o un guion bajo literales, hay que declarar un carácter de escape con la cláusula ESCAPE: LIKE '100!%' ESCAPE '!' busca el texto 100% tal cual.

```
cursor.execute("SELECT nombre FROM productos WHERE nombre LIKE ?", ("Mo____",))  
print(cursor.fetchall())
```

Salida

```
[('Monitor',)]
```

Documentación: [Expresiones](#)



El valor NULL

NULL significa dato ausente, no cero ni cadena vacía. Cualquier comparación con **NULL** mediante el operador de igualdad (=) devuelve **NULL**, nunca verdadero.

Nota técnica. Como WHERE solo deja pasar las filas cuya condición es verdadera, y NULL no lo es, una fila con NULL nunca se selecciona con una comparación normal. Ni siquiera `NULL = NULL` es verdadero.

```
cursor.execute("SELECT nombre FROM vendedores WHERE region = ?", (None,))  
print(cursor.fetchall())
```

Salida

[]



El valor NULL

Para preguntar por ausencia de valor se usan los operadores **IS NULL** e **IS NOT NULL**. No admiten marcador de posición, porque forman parte de la sintaxis y no son un valor.

```
cursor.execute("SELECT nombre FROM vendedores WHERE region IS NULL")
print(cursor.fetchall())
```

Salida

```
[('Pedro Rojas',)]
```

Documentación: [Manejo de NULL](#)



Ordenar y limitar

La cláusula **ORDER BY** ordena el resultado. La palabra **ASC** ordena de menor a mayor y es el comportamiento por omisión; **DESC** ordena de mayor a menor. La cláusula **LIMIT** restringe la cantidad de filas.

Nota técnica. Sin **ORDER BY**, el orden de las filas no está garantizado. Puede parecer estable durante meses y cambiar al añadir un índice, porque depende de cómo el planificador decida recorrer los datos.



Ordenar y limitar

```
cursor.execute("""
    SELECT nombre, precio FROM productos
    ORDER BY precio DESC
    LIMIT 2
""")
print(cursor.fetchall())
```

Salida

```
[('Laptop', 2500.0), ('Monitor', 1200.0)]
```

Documentación: [SELECT](#)



Funciones de agregación

Una función de agregación calcula un solo valor a partir de muchas filas. Son cinco las de uso corriente, y cada una se recorre por separado.

Documentación: [Funciones de agregación](#)



COUNT

Cuenta filas. Con el asterisco (*) cuenta todas las del grupo.

```
cursor.execute("SELECT COUNT(*) FROM ventas")  
print(cursor.fetchone()[0])
```

Salida

12

Documentación: [Funciones de agregación](#)



SUM

Suma los valores de una columna numérica.

Nota técnica. SUM() devuelve NULL, no cero, cuando no hay ninguna fila que sumar. La función TOTAL() hace lo mismo pero devuelve 0.0 en ese caso, y es específica de SQLite.

```
cursor.execute("SELECT SUM(cantidad * precio) FROM ventas")
print(cursor.fetchone()[0])
```

Salida

21600.0

Documentación: [Funciones de agregación](#)



AVG

Calcula el promedio aritmético de una columna numérica.

Nota técnica. AVG() divide entre la cantidad de valores no nulos, no entre el total de filas. Con datos ausentes el resultado difiere de dividir SUM() entre COUNT(*).

```
cursor.execute("SELECT AVG(precio) FROM ventas")  
print(round(cursor.fetchone()[0], 2))
```

Salida

1250.0

Documentación: [Funciones de agregación](#)



MIN

Devuelve el valor más bajo de la columna.

Nota técnica. MIN() con un solo argumento es una función de agregación y recorre las filas. Con dos o más argumentos es otra función distinta, escalar, que compara valores dentro de una misma fila.

```
cursor.execute("SELECT MIN(precio) FROM ventas")  
print(cursor.fetchone()[0])
```

Salida

45.0

Documentación: [Funciones de agregación](#)



MAX

Devuelve el valor más alto de la columna.

Nota técnica. Al usar MIN() o MAX() junto a otras columnas sin agrupar, SQLite sí devuelve los valores de la fila que produjo ese extremo. Es la única excepción documentada a la regla de las columnas fuera del grupo.

```
cursor.execute("SELECT MAX(precio) FROM ventas")  
print(cursor.fetchone()[0])
```

Salida

2500.0

COUNT(*) cuenta filas. **COUNT(columna)** cuenta solo los valores no nulos de esa columna.



MAX

Nota técnica. Todas las funciones de agregación salvo COUNT(*) ignoran los NULL. Por eso AVG() promedia sobre los valores presentes, no sobre el total de filas, y puede dar un resultado distinto al esperado.

```
cursor.execute("SELECT COUNT(*), COUNT(descuento) FROM ventas")  
print(cursor.fetchone())
```

Salida

(12, 9)

Documentación: [Funciones de agregación](#) · [Manejo de NULL](#)



Agrupar con GROUP BY

La cláusula **GROUP BY** divide las filas en grupos según el valor de una columna, y la función de agregación se calcula dentro de cada grupo.

```
cursor.execute("""
    SELECT categoria, SUM(cantidad * precio) AS total
    FROM ventas
    GROUP BY categoria
""")
for fila in cursor.fetchall():
    print(fila["categoria"], fila["total"])
```



Agrupar con GROUP BY

Salida

Accesorios 1350.0

Electronica 18700.0

Oficina 1550.0

Documentación: [SELECT](#) · [Funciones de agregación](#)



Filtrar grupos con HAVING

La cláusula **WHERE** filtra filas antes de agrupar. Para filtrar los grupos ya calculados se usa **HAVING**.

Nota técnica. El orden de evaluación es FROM, WHERE, GROUP BY, HAVING y por último SELECT. Por eso WHERE no puede usar el resultado de una agregación: cuando se evalúa, ese resultado todavía no existe.

```
cursor.execute("""
    SELECT categoria, SUM(cantidad * precio) AS total
    FROM ventas
    GROUP BY categoria
    HAVING total > 5000
""")
print(cursor.fetchall()[0]["categoria"])
```



Filtrar grupos con HAVING

Salida

Electronica

Documentación: **SELECT**



Unir tablas con JOIN

Un **JOIN** combina filas de dos tablas usando una columna que ambas comparten. La condición va tras la palabra **ON**.

Nota técnica. Las letras v y vt del ejemplo son alias de tabla: nombres cortos válidos solo dentro de esa consulta. Sirven para desambiguar cuando dos tablas tienen columnas con el mismo nombre.

```
cursor.execute("""
    SELECT v.nombre, vt.producto
    FROM ventas vt
    JOIN vendedores v ON vt.vendedor_id = v.id
""")
print(cursor.fetchall()[0]["nombre"])
```



Unir tablas con JOIN

Salida

Ana Quispe

Documentación: [Cláusula JOIN](#)



INNER JOIN y LEFT JOIN

El **JOIN** por omisión es **INNER**: devuelve solo las filas con coincidencia en ambas tablas, de modo que un vendedor sin ventas no aparece. **LEFT JOIN** conserva todas las filas de la tabla izquierda, que es la nombrada tras **FROM**.

Nota técnica. SQLite implementa INNER JOIN y LEFT JOIN desde siempre, y desde la versión 3.39 también RIGHT JOIN y FULL OUTER JOIN. Antes de esa versión, un RIGHT JOIN se escribía invirtiendo el orden de las tablas.



INNER JOIN y LEFT JOIN

```
cursor.execute("""
    SELECT v.nombre, COUNT(vt.id) AS num_ventas
    FROM vendedores v
    LEFT JOIN ventas vt ON vt.vendedor_id = v.id
    GROUP BY v.id, v.nombre
""")
for fila in cursor.fetchall():
    print(fila["nombre"], fila["num_ventas"])
```



INNER JOIN y LEFT JOIN

Salida

Ana Quispe 2

Luis Mendoza 1

Rosa Ttito 0

Nótese que se usa `COUNT(vt.id)` y no `COUNT(*)`. Con `COUNT(*)` el vendedor sin ventas contaría 1, porque el `LEFT JOIN` produce una fila para él con las columnas de la derecha en `NULL`.

Nota técnica. Es el error más frecuente al combinar `LEFT JOIN` con agregación. `COUNT(columna)` ignora los `NULL` y por eso devuelve el cero correcto.

Documentación: [Cláusula JOIN](#) · [Funciones de agregación](#)



Claves foráneas

Una clave foránea declara que una columna debe corresponder al identificador de otra tabla. Impide registrar una venta de un vendedor que no existe.

```
cursor.execute("""
    CREATE TABLE ventas (
        id            INTEGER PRIMARY KEY,
        vendedor_id   INTEGER NOT NULL REFERENCES vendedores(id),
        producto      TEXT     NOT NULL,
        cantidad      INTEGER NOT NULL
    )
""")
```



Claves foráneas

Documentación: [Claves foráneas](#)



Las claves foráneas están desactivadas por omisión

SQLite no verifica las claves foráneas salvo que se activen explícitamente en cada conexión.

Nota técnica. La verificación está desactivada por compatibilidad con versiones anteriores a la 3.6.19. El PRAGMA afecta solo a la conexión actual y no queda guardado en el archivo: hay que ejecutarlo cada vez que se abre la base.

```
conexion.execute("PRAGMA foreign_keys = ON")
```



Las claves foráneas están desactivadas por omisión

```
try:
    cursor.execute(
        "INSERT INTO ventas (vendedor_id, producto, cantidad) VALUES (?, ?, ?)",
        (999, "Laptop", 1),
    )
except sqlite3.IntegrityError as e:
    print("Rechazado:", e)
```

Salida

Rechazado: FOREIGN KEY constraint failed

Documentación: [Claves foráneas](#) · [Sentencias PRAGMA](#) · [IntegrityError](#)



Fin de la sesión

Consultas, agregación y JOIN

Alejandro Valentino Seminario Medina